Machine Learning Modelling Portfolio

By

Mary-Jane Masagbor

1. House Pricing Analysis Report

1.1 Introduction

The report analyses a housing dataset from King County to build models that predict sale prices using regression techniques. Two models are developed: a simple linear regression using only *sqft_living* and a multiple linear regression using a wider set of property features such as bedrooms, bathrooms, condition and grade. The aim is to understand the main factors influencing house prices, evaluate model performance and present findings clearly through visualisations.

1.2 Data processing and exploratory analysis

1.2.1 Data loading and structure

The dataset ho*useprice_data.csv* was loaded. Initial inspection with df.head(), df.info() and df.describe() provided an overview of the data types, ranges and summary statistics. The dataset contains a target variable price and multiple numerical and categorical predictors including:

- sqft_living, sqft_lot, sqft_above, sqft_basement, sqft_living15, sqft_lot15 – various measures of size
- bedrooms, bathrooms, floors – basic dwelling characteristics
- waterfront, view, condition, grade – quality and amenity indicators
- yr_built, yr_renovated – temporal variables
- lat, long, zipcode – geographic attributes

1.2.2 Feature selection

The model predictors were chosen by removing latitude, longitude, and postcode, as these location variables would require specialised spatial or categorical encoding methods. The target variable price was also removed from the predictor set. The remaining numerical features such as bedrooms, bathrooms, living area, condition, and grade, were retained because they are suitable for linear regression and provide meaningful structural and quality information about each property. This selection ensures the model uses relevant, well-behaved numerical inputs without additional preprocessing.

1.2.3 Train-test split

To obtain an unbiased estimate of predictive performance, the data was split into training and test sets.

- 80% of the records were used to **train** the models.

- 20% were held out as an independent **test** set.
- A fixed random state ensures that the split is reproducible.

1.3 Model Development

1.3.2 Simple linear regression: sqft_living

A simple linear regression model was created using *sqft_living* to predict house prices. This baseline model is easy to interpret, as the single coefficient reflects how price changes with additional living space. Although it captures an important relationship, its predictive power is limited because it does not account for other influential features such as quality, condition or number of bedrooms. The resulting R^2 score shows moderate explanatory ability, highlighting the need for a more comprehensive model.

This baseline demonstrates the value of a simple, interpretable model but also motivates the need for a richer specification.

1.3.3 Multiple linear regression

The model simultaneously considers multiple structural and quality attributes. The resulting coefficient vector quantifies the marginal effect of each feature while holding the others constant. For example, one coefficient represents the effect of an additional bathroom, another reflects the effect of improved grade, and so on.

Although *StandardScaler* was imported, scaling was not applied in the final code. For ordinary least squares without regularisation, this does not affect the predictions, but scaling could be beneficial if future work includes penalised models.

1.3.4 Model comparison

The two models are compared via their R^2 values:

- The simple model based on *sqft_living* explains a reasonable but limited proportion of the variance in price.
- The multiple linear regression model with all features yields a noticeably higher R^2, demonstrating that price is influenced by a combination of size, quality, condition and other factors, rather than a single attribute.

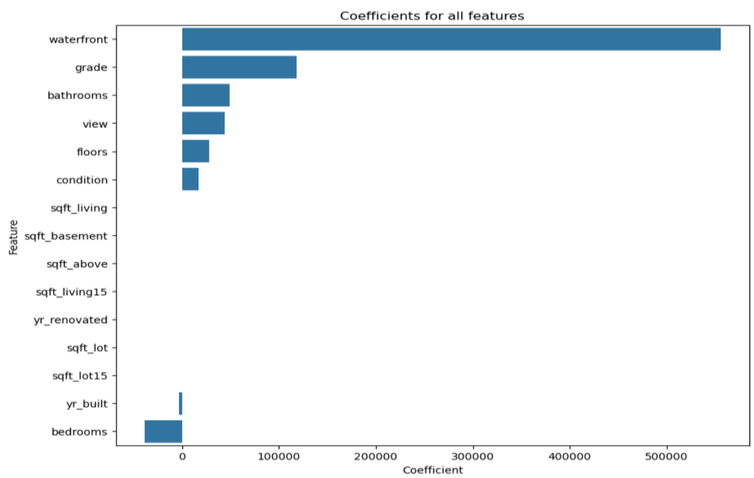
For a client, this means that relying on living area alone will under-estimate or over-estimate prices for properties with unusual combinations of features (e.g. small but high-grade homes, or large but poor-condition properties).

1.4. Visualisation and analysis of model performance

Several visualisations were produced to help interpret the fitted models and their behaviour. Three of them are particularly useful from a client perspective.

1.4.1 Feature importance: coefficient bar chart (Figure 1)

A bar plot of model coefficients was created:

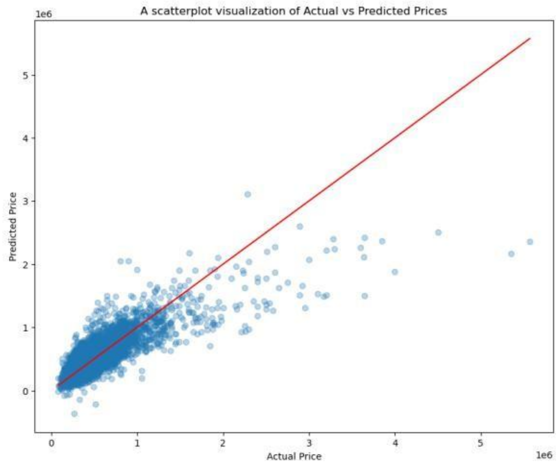


Interpretation:

- Features with large positive coefficients are associated with higher prices (e.g. greater living area, more bathrooms, higher grade).
- Features with negative coefficients reduce the predicted price (for example, older build year or certain size measures, depending on the exact estimates).
- This plot provides a clear ranking of which attributes matter most in the model, offering practical insight into which improvements may yield the greatest return on investment.

1.4.2 Actual vs predicted prices (Figure 2)

To assess predictive accuracy visually, a scatterplot of actual vs predicted prices on the test set was produced:



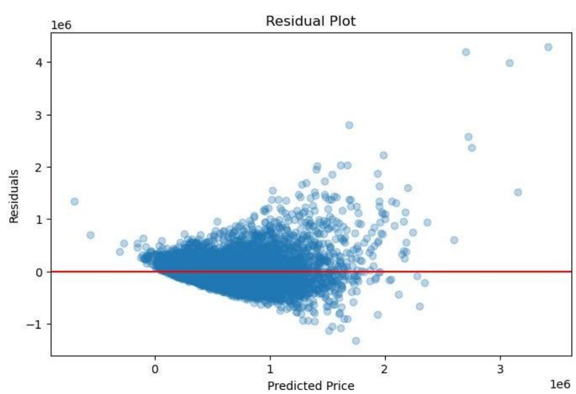
Interpretation:

- Each point represents a property; the closer it lies to the red 45° line, the closer the prediction is to the true sale price.
- The overall cloud of points is concentrated around the diagonal, indicating that the model is capturing the broad pricing structure of the market.
- Some dispersion and outliers remain, particularly at higher price levels, which is expected given the skewed distribution and the linear nature of the model.

This visualisation is intuitive for non-technical stakeholders: they can immediately see how closely the model aligns with reality.

1.4.3 Residual analysis (Figure 3)

Residuals (prediction errors) on the training set were computed:



Interpretation:

- Residuals are scattered around zero, which is consistent with the assumptions of linear regression.

- If the pattern remains roughly symmetric with no strong curvature, it suggests that the linear form is reasonably appropriate.

1.5. Conclusion and recommendations

1.5.1 Summary of findings

The simple regression model using only *sqft_living* provides a clear baseline but explains only part of the variation in house prices. The multiple linear regression model performs much better, capturing the combined effects of size, condition, quality and other structural features. Exploratory analysis shows strong relationships between price and variables such as grade and living area. Visual diagnostics confirm that the model fits well overall, though outliers and high-value properties introduce some error. Overall, the modelling pipeline is clear, effective and provides reliable insights into the factors driving property prices.

1.5.2 Recommendations

The analysis suggests prioritising improvements that enhance living space and overall quality, as these features strongly influence price. The multiple linear regression model should be used for valuation guidance because it captures a wider range of property attributes. Predictions for extremely high-value or unusual homes should be interpreted cautiously. Future enhancements could include log-transforming prices, applying regularised regression, adding non-linear terms and incorporating encoded location data. These steps would further strengthen accuracy and model reliability.

2. Clustering Analysis Report

2.1 Introduction

This report analyses a dataset of countries using clustering techniques to identify groups with similar socio-economic and health characteristics. Since there are no predefined labels, unsupervised learning—specifically K-means—is used to uncover natural segments. The aim is to support better policy targeting, investment planning and international development decisions. The work progresses from simple two-feature clustering to more complex, multi-dimensional models, supported by clear visualisations.

2.2. Data processing and exploratory analysis

2.2.1 Data loading and structure

The dataset *country_data.csv* was loaded and basic inspection was carried out (df.head(), df.inf0, df.describe, df.isna().sum()). These steps confirm the structure of the data, the types (numerical vs categorical) and whether there are missing values. The first column contains country names (a text identifier), while the remaining columns hold continuous numerical indicators related to health, demography, trade and macro-economic performance. No explicit missing-value handling was required, indicating that the dataset is complete for the variables used in clustering.

2.2.2 Preparing features for clustering

Because country names are identifiers rather than measurable attributes, the first column was removed before further analysis:

```
df_corr= df.drop(df.columns[0], axis=1)
df_corr.corr()
```

This subset, *df_corr*, contains only numerical features and is used for correlation analysis and scaling. After scaling, each feature has approximately zero mean and unit variance. This ensures that all indicators contribute comparably to the clustering process.

2.3. Model development

2.3.1 Clustering algorithm and scaling

The main algorithm used is K-means clustering. K-means partitions the dataset into Kclusters by iteratively:

1. Initialising K centroids.
2. Assigning each country to the closest centroid.
3. Updating each centroid to be the mean of the countries assigned to that cluster.

The algorithm stops when assignments stabilise or when a maximum number of iterations is reached. Because the features are standardised beforehand, each variable contributes proportionately to the distance calculation.

A fixed *random_state* (e.g. 42) is used in the code to ensure reproducible cluster assignments.

2.3.2 Initial 2-dimensional clustering (child mortality and income)

As an initial, easily interpretable model, clustering is performed using only two key indicators drawn from the scaled dataset. These dimensions are chosen because they capture:

- Health outcome (child mortality), and
- Economic capacity (income).

Before fitting K-means, a simple scatter plot is generated to show the distribution of countries in this 2D space. A K-means model with three clusters is then trained.

2.3.3 Intermediate model: clustering with additional dimensions

To capture more nuance, the clustering is extended to use a subset of four indicators. These additional features (for example, health expenditure and GDP per capita) enrich the description of each country beyond just mortality and income. A three-dimensional scatter plot is created to visualise projections of this higher-dimensional space. This intermediate model demonstrates how including more indicators alters the cluster boundaries and provides a more complete representation of socio-economic status.

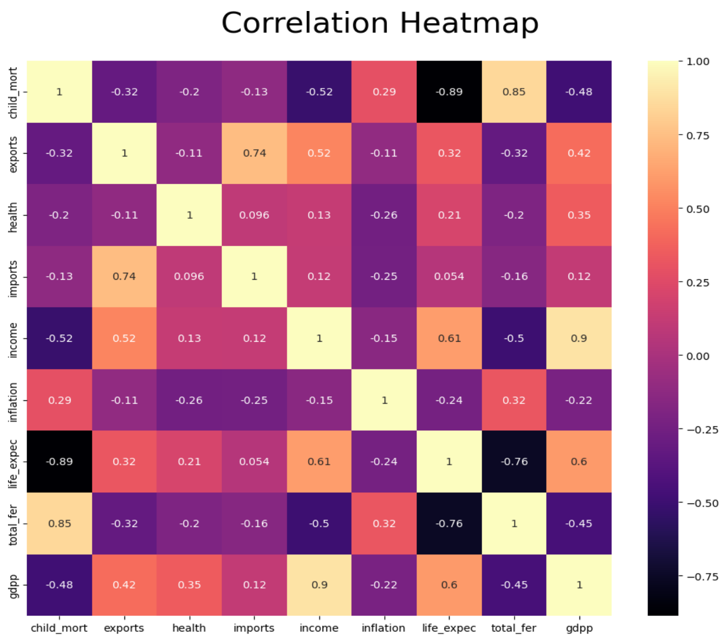
2.3.4 Final model: clustering on all available features

The most complex and informative model uses all standardised numerical features. Here, K-means operates in the full multi-dimensional feature space, jointly considering child mortality, exports, health expenditure, income, inflation, life expectancy, fertility, GDP per capita and any remaining indicators. Cluster sizes are inspected using *df.cluster.value_counts()* to check that the segmentation is not dominated by a single group.

2.4. Visualisation and analysis

Clear, client-friendly visualisations are produced at each stage to support interpretation of the clustering results.

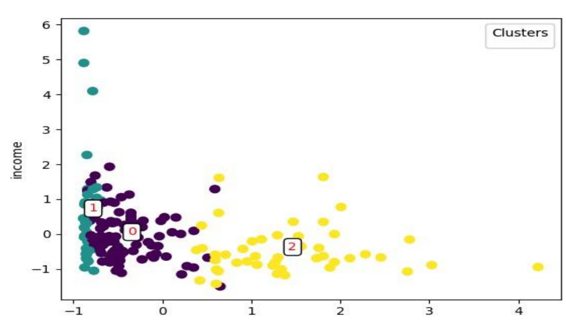
2.4.1 Correlation heatmap



This correlation heatmap provides an initial overview of relationships between variables. It reveals that:

- Countries with higher income and GDP per capita tend to have lower child mortality and higher life expectancy.
- High fertility and high child mortality often co-occur, particularly in lower-income economies.
- Trade indicators such as exports and imports are positively related to overall economic performance.

2.4.2 Two-dimensional cluster visualisation (child mortality vs income)

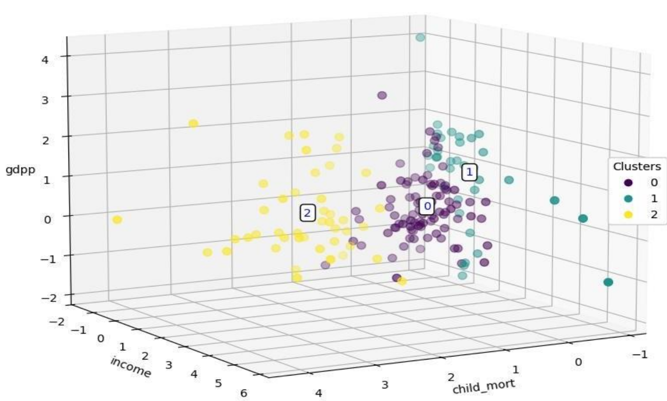


This visualisation shows:

- A cluster of high-income, low-mortality countries in one corner of the plot.
- A cluster of low-income, high-mortality countries in another.
- A middle band of countries with intermediate income and health outcomes.

The centroid labels (0, 1, 2) help recognise the typical position of each cluster in terms of health and income.

2.4.3 Three-dimensional cluster visualisation



Although the model itself operates in the full feature space, this projection illustrates how clusters separate when viewed through three key dimensions. Visually:

- The high-development cluster occupies the region with low child mortality, high income and high life expectancy.
- The low-development cluster is located where child mortality is high and income and life expectancy are low.
- The intermediate cluster lies between these extremes.

2.5. Conclusion and recommendations

2.5.1 Summary of findings

The clustering analysis groups countries into three clear segments based on health, demographic and economic indicators. The initial two-feature model highlights strong separation between high- and low-development countries. Using more features refines these groupings and captures greater nuance in development profiles. The elbow method supports three clusters as the most balanced choice. Overall, the modelling process is clear, reproducible and aligns well with known global development patterns.

2.5.2 Recommendations

The three clusters can be used as broad development bands to guide targeted policy decisions. Key indicators such as child mortality, income and life expectancy should be prioritised when planning interventions. Countries within the same cluster can be benchmarked against one another to identify effective practices. More detailed or region-specific clustering can be carried out when needed. Future improvements may include testing additional cluster numbers, alternative algorithms or dimensionality-reduction methods.

3. Predicting NBA Rookie Longevity

3.1. Introduction

This report uses NBA rookie performance data to predict whether a player will remain in the league for at least five years. This is treated as a supervised binary classification task. A range of models such as Logistic Regression, Gaussian Naïve Bayes and neural network architectures are developed and compared. The aim is to build a progression from simple linear classifiers to more complex neural networks, evaluate their performance, and present findings through clear visualisations.

3.2. Data processing and exploratory analysis

3.2.1 Data loading and cleaning

The dataset was loaded, and missing rows were dropped (*df = df.dropna()*). This ensures all subsequent models receive complete cases only. The resulting data frame contains 21 columns:

- 1 identifier – Name
- 19 numerical performance statistics – including Games Played, Minutes Played, Points Per Game, Field Goal Percent, 3 Point Percent, Free Throw Percent, Rebounds, Assists, Steals, Blocks, Turnovers, etc.
- 1 binary target – TARGET_5Yrs

The target column encodes whether the player's career length is at least five seasons.

3.2.2 Feature set and encoding

For the classical models (Logistic Regression and Gaussian Naïve Bayes), the name column is removed so that clustering depends only on performance attributes.

Within *df_corr*, the last column is TARGET_5Yrs. Feature subsets used in the experiments include:

- **2-feature subset**: Games Played C Points Per Game
- **4-feature subset**: Games Played, Minutes Played, Points Per Game, C Field Goals Made
- **Full feature set**: All performance statistics except TARGET_5Yrs.

For the neural networks, all non-numeric columns (principally Name) are encoded via LabelEncoder so that they can be used within scikit-learn's MLPClassifier. This gives a fully numerical feature matrix suitable for neural network training.

3.2.3 Train-test split and scaling

To estimate generalisation performance, the data is split into training and test sets:

- For the classical models, an 80/20 split is used without stratification.
- For the neural networks, a stratified 80/20 split is used to preserve the proportion of long-career and short-career players in each subset.

Because the features are on different scales (for example, percentages vs counts), standardisation is applied before training models that are sensitive to feature scale (Logistic Regression, GaussianNB when used with continuous features, and neural networks).

3. Model development

3.3.1 Overview of algorithms and evaluation

Three main model families are implemented:

1. **Logistic Regression**
    - A linear probability classifier using the logistic (sigmoid) function to map a weighted sum of inputs to a probability of TARGET_5Yrs = 1.
    - Implemented via LogisticRegression with solver='lbfgs' and a maximum of 2000 iterations for reliable convergence.
2. **Gaussian Naïve Bayes (GNB)**
    - A probability classifier that assumes features are conditionally independent given the class, and normally distributed.
    - Implemented with scikit-learn's GaussianNB.
3. **Multi-Layer Perceptron (MLP) neural networks**
    - Fully connected feed-forward neural networks trained with backpropagation.
    - Implemented with scikit-learn's MLPClassifier using various hidden layer sizes and activation functions (logistic, ReLU, tanh).

3.3.2 Baseline models: Logistic Regression and Gaussian Naïve Bayes

(a) 2-feature models: games and points

For the simplest setting, only two intuitive features are used. After splitting and scaling, a logistic regression model is trained. The model achieves test accuracy in the mid-70% range, misclassifying a few dozen rookies out of the test subset.

A Gaussian Naïve Bayes model performs slightly worse, reflecting the fact that the independence and Gaussian assumptions are not strictly satisfied and that the linear decision boundary of logistic regression is better aligned with the data in this low-dimensional space.

(b) 4-feature models: extended core statistics

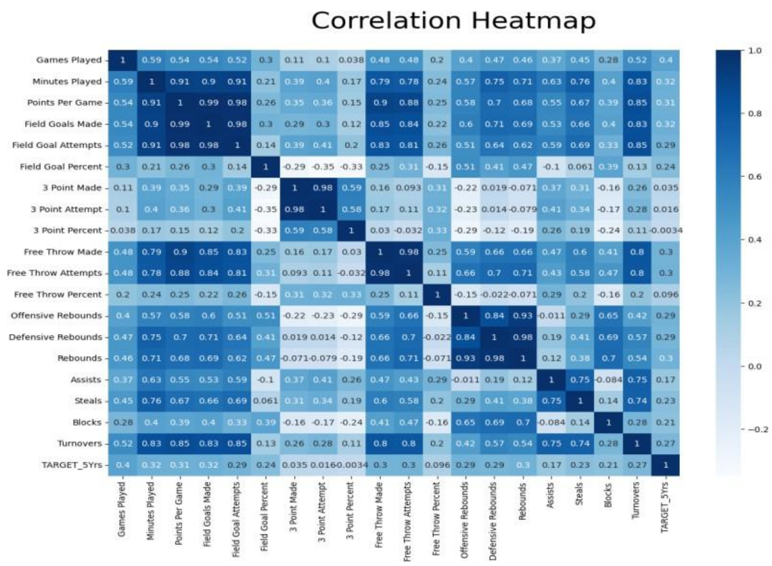
The four-feature model expands the baseline by adding minutes played and field goals made to games played and points per game. Logistic Regression trained on these features shows an improvement over the simpler two-feature model, demonstrating that increased playing time and scoring efficiency add predictive value. Gaussian Naïve Bayes is also applied but performs less consistently, as the strong correlations among these variables violate its independence assumptions. Overall, the four-feature setup provides a stronger, more informative baseline for predicting long-term NBA career outcomes.

3.3.3 Neural network models

1. All features, logistic activation
    - With hidden layers: this model has three hidden layers with increasing width, enabling it to learn non-linear combinations of the rookie statistics.
    - Without hidden layers: This effectively reduces to a single logistic layer, similar in spirit to logistic regression but optimised via the MLP framework.
2. All features, ReLU activation with hidden layers: ReLU activation allows the network to represent piecewise-linear boundaries and often trains faster and more robustly on deeper architectures.
3. All features, tanh activation with hidden layers: The tanh non-linearity is symmetric around zero and can model smooth non-linear boundaries, though it may be more prone to saturation than ReLU.

3.4. Visualisation and analysis

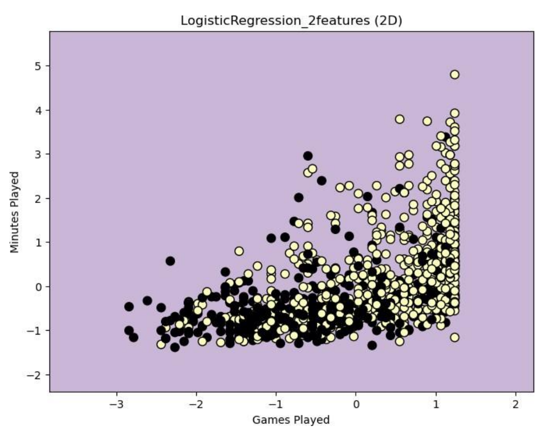
3.4.1 Correlation heatmap

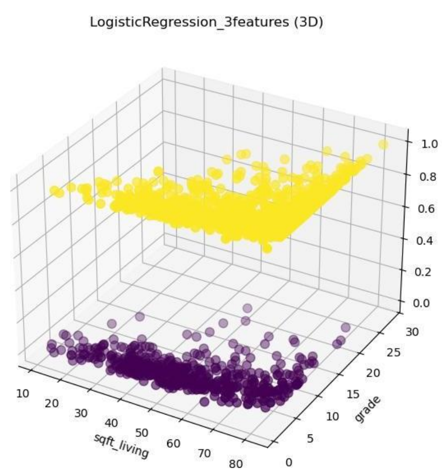


This heatmap highlights that:

- Players who play more games and minutes and contribute across the box score (points, rebounds, assists) are more likely to remain in the league for at least five years.
- High shooting percentages and efficient scoring are associated with better career outcomes.
- Certain statistics are strongly correlated with one another (for example, points and field goals made), which helps explain why model performance does not always improve by simply adding more features.

3.4.2 Logistic regression decision boundary (2D and 3D)



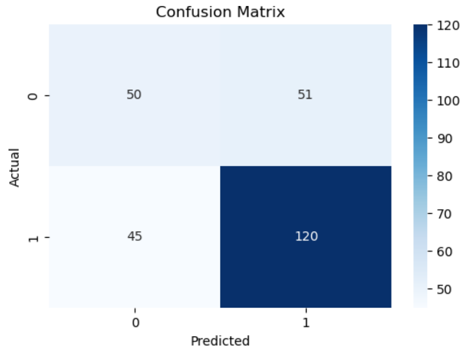


This visual clearly illustrates the linear boundary between “likely to last 5 years” and “unlikely” in this simplified feature space. The client can, for example, see that rookies with both high playing time and solid scoring tend to fall in the long-career region.

In higher dimensions (when more features are used), a 3D scatter plot is used to project three selected features and colour the points by model prediction. While the full decision surface lies in higher-dimensional space, this gives an intuitive sense of how performance characteristics combine to influence the model's decision.

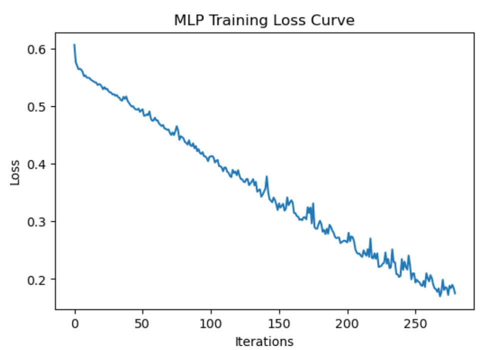
3.4.3 Neural network diagnostics: confusion matrix and loss curve

Confusion matrix heatmap



This matrix shows how many rookies in each true class are predicted as long- or short-career. It allows the client to see, for example, whether the model tends to miss borderline success stories (false negatives) or over-optimistically project success (false positives).

Training loss curve



The loss curve illustrates how the network's training error decreases over iterations. A smoothly decreasing curve that flattens out indicates stable training and convergence; sharp oscillations or divergence would suggest problems with learning rate or network design.

Together, these visualisations demonstrate not only how well the models perform but also **how** they arrive at their predictions and whether the training process is well-behaved.

3.5. Conclusion and recommendations

3.5.1 Summary of findings

The analysis shows that predicting NBA career longevity is feasible using rookie-season statistics. Logistic Regression offers strong, reliable baseline performance, while Gaussian Naïve Bayes is weaker due to correlated features. Neural networks with hidden layers can model more complex relationships and occasionally outperform linear models, particularly when using all available features. Visual tools—such as heatmaps, decision boundaries and confusion matrices—help explain both the data patterns and model behaviour. Overall, the modelling pipeline is effective, interpretable and suitable for supporting evidence-based scouting decisions.

3.5.2 Recommendations

Logistic Regression is recommended as the primary model because it is accurate, interpretable and reliable. Neural networks can be used when marginal performance gains are needed and interpretability is less critical. Model predictions should support, not replace, human scouting, and confusion matrices should guide how risk-tolerant the decision process should be. Adding more features, using regularisation and applying cross-validation can further improve performance. Overall, the models provide a strong foundation for data-driven decisions about rookie potential.

4. Ethics of AI – The Trolley Problem and Autonomous Vehicles

The Trolley Problem is a classic ethical dilemma where a person must decide whether to divert a runaway trolley to save several people at the cost of killing one. For years it has been mainly a philosophical exercise, but autonomous vehicles have turned it into a real and uncomfortable design problem. Unlike humans, who react instinctively under pressure, autonomous cars must follow rules written in advance. This means moral decisions that were once hypothetical now have to be programmed into actual machines.

In the context of autonomous vehicles, the Trolley Problem arises when a collision is unavoidable. For example, should the vehicle swerve to save a group of pedestrians even if doing so risks killing its own passenger? Or should it prioritise the person inside the car because they entrusted their safety to the machine? These questions highlight a major challenge: it is no longer about what a person chooses in a split second, but what a system is instructed to do—and who carries responsibility for that decision [3].

One of the biggest difficulties is that there is no universally accepted moral framework that engineers can simply "translate" into code. A strictly utilitarian system might favour saving the greatest number of people, even if it means sacrificing its passenger. But this approach creates practical and commercial problems. Most people would be reluctant to buy a vehicle programmed to sacrifice them for the greater good [1]. On the other hand, a vehicle that always prioritises its passenger could put pedestrians and cyclists at greater risk, creating an unfair distribution of harm in public spaces.

Another issue is fairness and potential bias. Autonomous vehicles rely on data to learn patterns, but datasets may reflect social inequalities. If an algorithm is trained on biased or incomplete data, its split-second decisions could unintentionally discriminate—for example, by reacting differently to people in different neighbourhoods or with different physical characteristics. This raises the question of whether the Trolley Problem is really about one dramatic decision, or about the quieter, everyday decisions that an autonomous vehicle makes long before an accident occurs [2].

Regulation therefore becomes crucial. It is unrealistic and unfair to expect private companies alone to decide who should be protected and when. Clear national or international guidelines are needed to ensure that all autonomous vehicles follow transparent and ethically defensible rules. Some researchers argue that the best long-term approach is to design systems that minimise the chance of a trolley-type scenario in the first place by focusing on safer driving behaviour, more cautious risk estimation and improved sensing technology.

In summary, the Trolley Problem shows that autonomous vehicles force society to confront the uncomfortable reality of moral decision-making in technology. The goal should not be to find a perfect moral answer, but to create clear, accountable and fair frameworks for how autonomous vehicles behave. By doing this, we can ensure that ethical thinking remains at the centre of technological progress.

References

[1] Bonnefon, J-F., Shariff, A. and Rahwan, I. (2016) 'The Social Dilemma of Autonomous Vehicles', *Science*,352(6293),pp.1573–1576.
[2] Calo, R. (2020) 'Artificial Intelligence Policy: A Primer and Roadmap', *University of California Press*.
[3] Lin, P. (2016) 'Why Ethics Matters for Autonomous Cars', *Oxford University Press*.